Name: Praisel Ekpenyong (2025/A/CYB/0205)
Email: p.ekpenyong8162@miva.edu.ng
Department: Cybersecurity
Matric Number: 2025/A/CYB/0205

COS202 - Computer Programming II - Lab Assessment

Part 1: Source Files/Code

The complete C++ source is hosted on GitHub
https://github.com/praisel-ekpenyong/COS202_SCAPMS

i. Header Files

- **model/PatientRecord.h** — Base patient class.
- **model/GeneralPatient.h** — General outpatient class.
- **model/EmergencyPatient.h** — Emergency triage patient class.
- **model/FollowUpPatient.h** — Follow-up patient class.
- **model/Appointable.h** — Appointment scheduling interface.
- **model/Appointment.h** — Appointment data class.
- **model/AppointmentStatus.h** — Appointment status Enum.
- **model/PatientCategory.h** — Patient category Enum.
- **model/ClinicDatabase.h** — In-memory clinic database and cache.
- **controller/ClinicManager.h** — Core clinic workflow manager.
- **controller/SearchEngine.h** — Linear, binary, and recursive search.
- **controller/PatientSorter.h** — Selection sort and merge sort.
- **controller/SortAlgorithms.h** — Template sorting implementations.
- **controller/ReportEngine.h** — Clinic reporting and text export.
- **controller/UndoableAction.h** — Administrative undo action structures.
- **utils/Validators.h** — Input validation utilities.
- **utils/IDGenerator.h** — Auto-incrementing ID generator.
- **utils/FileHandler.h** — CSV file saving and loading.
- **utils/Exceptions.h** — Custom exception classes.
- **utils/AppInfo.h** — Application metadata definitions.

ii. Source Files

- **model/PatientRecord.cpp** — Base patient implementation.
- **model/GeneralPatient.cpp** — General patient implementation.
- **model/EmergencyPatient.cpp** — Emergency patient implementation.
- **model/FollowUpPatient.cpp** — Follow-up patient implementation.
- **model/Appointment.cpp** — Appointment implementation.
- **model/ClinicDatabase.cpp** — Database storage and cache implementation.
- **controller/ClinicManager.cpp** — Clinic manager logic and undo stack.
- **controller/SearchEngine.cpp** — Search algorithms implementation.
- **controller/PatientSorter.cpp** — Sorting algorithms implementation.
- **controller/ReportEngine.cpp** — Report generation logic.

- controller/UndoableAction.cpp — Undoable action implementations.
- utils/Validators.cpp — Form validation rules implementation.
- utils/IDGenerator.cpp — ID persistence implementation.
- utils/FileHandler.cpp — CSV reading and writing implementation.
- utils/AppInfo.cpp — Application metadata implementation.

iii. GUI Files and Resources

- gui/MainWindow.h / MainWindow.cpp — Main window and tab navigation.
- gui/DashboardPanel.h / DashboardPanel.cpp — Overview dashboard and statistics.
- gui/PatientPanel.h / PatientPanel.cpp — Patient registration and table.
- gui/AppointmentPanel.h / AppointmentPanel.cpp — Appointment scheduling and filters.
- gui/QueuePanel.h / QueuePanel.cpp — Priority triage waiting queue.
- gui/SearchPanel.h / SearchPanel.cpp — Record searching and sorting panel.
- gui/ReportPanel.h / ReportPanel.cpp — Clinic reports viewing and export.
- gui/AdminPanel.h / AdminPanel.cpp — Administrative undo and action log.
- gui/Theme.h / Theme.cpp — Application styling and color palette.
- gui/main.cpp — Application entry point.
- gui/resources.qrc — Application icon resource bundle.

iv. Data Files Required to Run the Application

- data/patients.csv — Sample patient records.
- data/appointments.csv — Sample appointment records.
- data/counters.csv — Persistent ID counter file.

Build and run instruction

```
# Configure project
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release

# Build application binary
cmake --build build

# Launch application
./build/gui/SCAPMS.exe
```

Part 2: Project Report

i. Project Description

The Smart Clinic Appointment & Patient Management System (SCAPMS) is a desktop application written in C++17 and Qt 6 Widgets. It manages patient records, handles appointment booking and tracking, and prioritizes waiting queues. It also provides custom search and sort routines, clinic report exports, and multi-level undo for administrative actions. All records are saved to CSV files. A pre-populated clinic dataset loads automatically at launch, so every panel can be tested right away.

The code is organized into four namespaces:

- **scapms::model:** defines the core patient, appointment, and database classes.
- **scapms::controller:** contains the business logic, sorting and searching routines, reporting calculations, and undo history.
- **scapms::gui:** contains the main window and seven specialized panel widgets.
- **scapms::utils:** provides validation, sequential ID generation, and file I/O.

The complete source code repository is publicly available at
https://github.com/praisel-ekpenyong/COS202_SCAPMS

ii. System features

The application covers all required clinic workflow features through dedicated interface panels:

- **Patient registration:** register, edit and delete records, with automatic P-xxxx IDs and validated names, ages and contact details.
- **Appointment:** Schedule, cancel and remove appointments, filter them by date and status, and see overdue visits flagged in red.
- **Priority queue:** Patients check into a consultation waiting queue. While the queue generally operates on a first-come, first-served basis, emergency cases are automatically moved ahead of general and follow-up patients. If a patient waits longer than 15 minutes, a visual alert triggers on the dashboard.
- **Record search:** Records can be looked up by ID, name prefix, or patient category. The search engine dynamically selects linear, binary, or recursive search depending on data state and reports the technique used directly to the status bar.
- **Record sorting:** Patient lists can be sorted by name, ID number, age, or upcoming appointment date using custom implementations of selection sort or merge sort, selectable via dropdown.
- **Undo:** reverse patient deletions, patient edits and appointment removals, most recent first.
- **Analytical reporting:** The system generates four reports (frequently attended patients, pending appointments, category breakdown, and daily summary) that can be viewed in tables or exported directly to plain text files.
- **Data storage:** Patients, appointments and ID counters are saved to and loaded from CSV files.

iii. Class hierarchy

PatientRecord is the abstract base class that holds the data every patient share: ID, full name, age, phone number and a category Enum. It declares two pure virtual methods, **describe()** and **priority()**, which provide runtime polymorphism. Three concrete classes inherit from this base:

- **GeneralPatient** has a baseline priority of 1.
- **FollowUpPatient** has a medium priority of 2 and adds a next-visit date field.
- **EmergencyPatient** has a high priority of 3.

Each subclass overrides **describe()** to return a summary specific to its category.

To handle appointment capabilities cleanly, I wrote an abstract interface called `Appointable`. It defines pure virtual methods including **scheduleAppointment(), cancelAppointment()**, **isScheduled(),** and **currentAppointmentId()**. Each concrete patient subclass implements this interface. Separating appointment operations into an interface keeps patient demographic data separate from scheduling state, meaning classes interact with appointment booking capabilities through clean abstractions.

Runtime polymorphism is demonstrated by functions such as describeThroughBase(const PatientRecord&) in the GUI and by ClinicManager::callNext(), which compares priority() through base-class pointers. The correct override runs for each patient type without the caller knowing that type. I use dynamic_cast only where a capability is not part of the base class: Appointable and the follow-up next-visit date.

ClinicDatabase demonstrates composition by owning patients and appointments through std::unique_ptr. Each Appointment stores its patient's ID, so one patient can have many appointments, retrieved with appointmentsFor(). ClinicManager coordinates the application and works with SearchEngine, PatientSorter and ReportEngine. Undo operations form a second hierarchy under the abstract UndoableAction class.

iv. Data structures used

I chose each data structure to suit the needs of the feature that uses it:

- **std::vector:** stores patients and appointments in ClinicDatabase. It grows as needed and gives the constant-time indexing that binary search and merge sort rely on.
- **std::queue:** holds the IDs of waiting patients in arrival order (FIFO), with a std::map storing each arrival time. callNext() empties the queue, picks the highest-priority patient, and puts the others back in their original order.
- **std::stack:** in ClinicManager stores undo actions. Before staff delete a patient, update patient details or remove an appointment, a snapshot of the current state is pushed onto the stack. Undo pops the latest snapshot and reverses that change.
- **Fixed-size array (PatientRecord* [10]):** ClinicDatabase keeps a 10-element array as a quick-access cache of the ten most recently used patient records. When staff view or edit a record, its pointer moves to index 0 and older entries shift down. If the array is full, the oldest entry is dropped.

v. Searching algorithms used

I wrote all the searches by hand in SearchEngine, without std::find or std::lower_bound.

- **Linear search (O(n))** checks records one at a time. It handles ID and name lookups on unsorted data, and category filtering.
- **Binary search (O(log n))** runs on a sorted copy of the records, halving the search range each step. It is used for ID and name lookups once the records have been sorted by that field.
- **Recursive search (O(n))** finds a patient by name when the user selects it (see section vii).

The technique depends on the state of the data. ClinicManager increases a revision counter on every change, and the sorted copy records the revision it was made from. If the two differ, the search falls back to linear. The technique actually used is shown in the status bar.

vi. Sorting algorithms used

I wrote both sorting algorithms as generic functions in `controller/SortAlgorithms.h` and users select them from a dropdown.

- **Selection sort** repeatedly finds the smallest remaining item and swaps it into place. It takes O(n²) comparisons, at most n−1 swaps and O(1) extra memory, and it is not stable.
- **Merge sort** recursively splits the list, then merges the sorted halves using one scratch buffer. It takes O(n log n) time on all inputs and O(n) extra memory, and it is stable because the left item is taken first on ties.

Records can be sorted by name, ID, age or next appointment date. Sorting works on a copy holding pointers, so the database order does not change.

vii. Recursive algorithm used

I used recursion for the name search in SearchEngine, which supports exact and prefix matching. The findByNameRecursive function puts the patient pointers in a vector, then calls a helper function, recursiveNameScan, starting at index 0.
The helper checks one patient at a time and has two base cases:

- **No match:** If the index reaches the end of the vector, it returns nullptr.
- **Match found:** If the patient's name matches the search, it returns a pointer to that patient.

In the recursive step, the helper calls itself with index + 1 to check the next patient. findByNameRecursive then returns a SearchResult containing the first matching patient, or an empty result if no patient matched.

viii. GUI design

MainWindow has a menu bar, a toolbar, a status bar and a QTabWidget with seven tabs: Dashboard, Patient Records, Appointments, Waiting Queue, Search & Sort, Reports and Administration. I chose these layouts for the following reasons:

- **QVBoxLayout and QHBoxLayout** stack each panel's sections and place buttons in rows, and they resize with the window.
- **QFormLayout** suits the patient form and appointment dialog because it lines up each label with its input.
- **QGridLayout** suits the dashboard summary and search controls because they fall naturally into rows and columns.

The program is event-driven through Qt signals and slots. These respond to button clicks, menu actions, combo box changes, table selection and pressing Enter in the search box. Two QTimers run every 30 seconds to update dashboard reminders and check waiting times.

I also implemented these features:

- **Custom colours:** red, green and slate text for emergency, follow-up and general patients, and row tints in the queue.
- **Dynamic components:** the next-visit field appears only for follow-up patients.
- **File dialogs:** used for choosing data folders and exporting reports.
- **Keyboard shortcuts:** Ctrl+S, Ctrl+F, Ctrl+Q and Ctrl+Z.
- **Tooltips:** added to the main buttons and controls.
- **Validation messages:** clear messages for invalid input, and a confirmation before deleting a record.

ix. Exception handling approach

I implemented custom exception handling in utils/Exceptions.h by defining three classes derived from std::runtime_error: ValidationError (for invalid input values), DuplicateIdError (for repeated IDs), and FileError (for file problems).

The application applies validation rules before mutating state. In `ClinicManager`, methods like `registerPatient` and `scheduleAppointment` validate names, age limits, phone numbers, and date formats before creating objects. If any validation fails, the method throws a `ValidationError` with a specific message. In the GUI layer, user actions are enclosed in try-catch blocks to display user-friendly warnings rather than letting the application crash:

When loading files, a bad row is skipped and reported in a summary dialog while valid rows still load. A FileError shows a warning, or starts the program empty if it occurs at startup.

xi. Algorithms chosen and reasons

I selected two sorting algorithms and three searching algorithms for SCAPMS, choosing each to fit a specific operational requirement in the clinic:

Sorting Algorithms

- **Merge sort** is used for the reports and suits larger patient lists because its running time stays O(n log n). Its stability keeps patients with the same age or date in their original order.
- **Selection sort** suits small lists because it is simple, uses no extra buffer and makes few swaps. I also use it to order the appointment table by date and time.

Searching Algorithms

- **Linear search:** Chosen for unsorted data. Clinic staff add and edit records throughout the day, so sorting the whole database after every change would be unnecessary. Linear search does not need sorted data, so it handles ID and name lookups when no up-to-date sorted view is available, as well as category filtering.
- **Binary search** Chosen for fast ID or name lookups after the records have been sorted by that field. A revision check confirms that the sorted copy is still current, and the search falls back to linear if the data has changed.
- **Recursive search**: Chosen for exact and prefix name searches to demonstrate recursive problem solving. Instead of scanning with a loop, the helper function checks one patient, then calls itself for the next. It stops at a base case when it finds a match or reaches the end of the list.

xii. Challenges encountered

During the development and testing of SCAPMS, I ran into several practical programming challenges:

1. I encountered intermittent application crashes when deleting a patient who was also present in the 10-slot recently accessed patient cache (`recentPatients[10]`). The cache held raw non-owning pointers to `PatientRecord` instances. When a patient was deleted, `ClinicDatabase` freed the underlying object, but the cache slot still held the deallocated memory address. Clicking on that entry from the dashboard triggered a segmentation fault.
2. My first Merge Sort implementation had off-by-one errors when splitting sub-arrays, and these appeared mainly with odd element counts. Incorrect boundaries could produce a range that never got smaller, leading to infinite recursion, while merging without a separate buffer overwrote values that had not yet been copied.
3. After a user sorted patients by name, name searches switched to Binary Search over the sorted copy. If the user then added a patient or edited a name, that copy was out of date: a new patient was missing from it, and an edited name could be out of order. As a result, searches could miss existing records.

4. An Undo refreshes every panel, and rebuilding the patient table changed its selection programmatically. This fired Qt's itemSelectionChanged signal, which ran the handler meant for user clicks, reloading the form and switching it into edit mode.

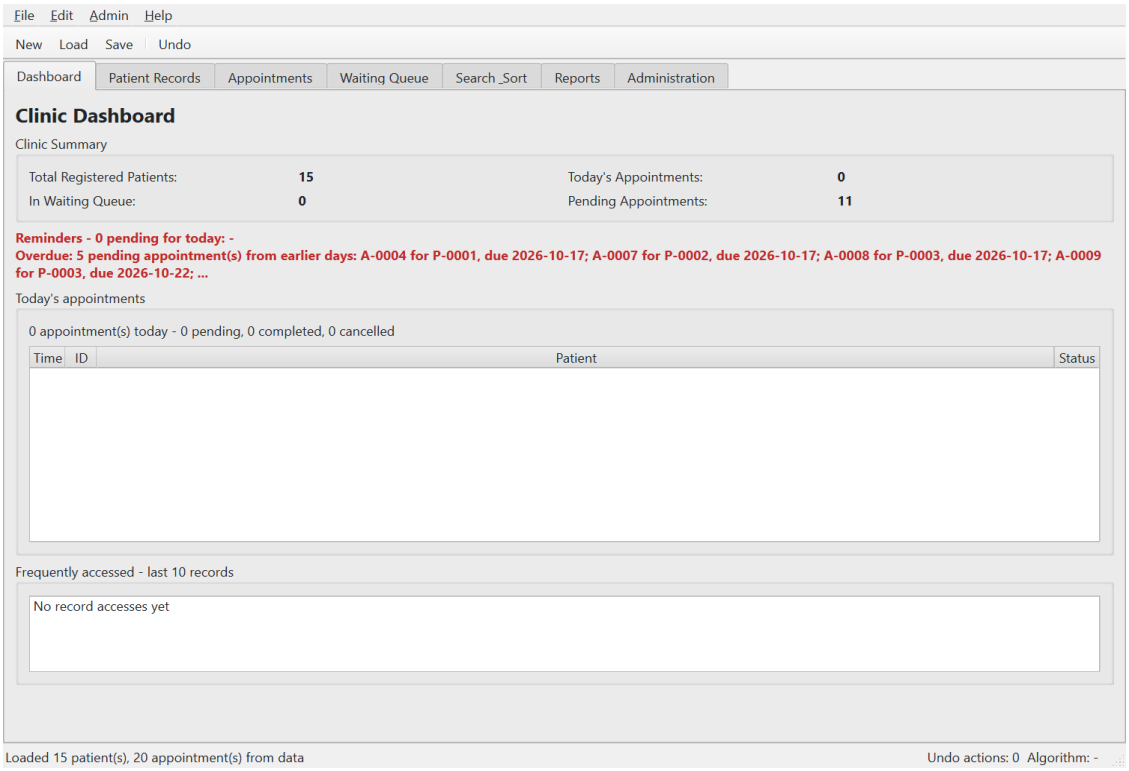
xii. How the challenges were resolved

I resolved these development issues through targeted design and debugging adjustments:

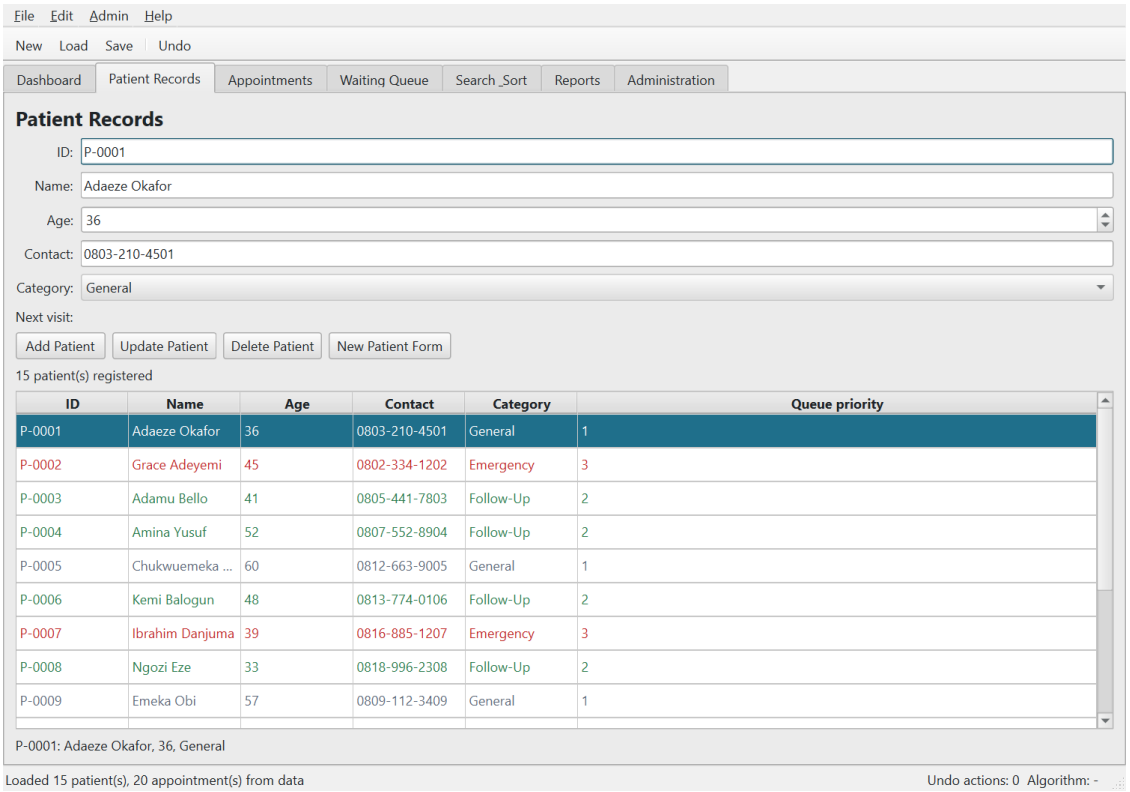
1. To resolve the dangling pointer crash in the cache, I added a `forgetRecent(const std::string& patientId)` method to `ClinicDatabase`. Now, whenever `deletePatient()` is invoked, the database scans the `recentPatients` array, nullifies any matching pointer, and compacts remaining entries forward before releasing the object from memory.
2. To fix the merge sort errors, I restructured the algorithm in SortAlgorithms.h into two functions. mergeSortRange divides the list using half-open ranges, [lo, mid) and [mid, hi), which made the boundaries consistent for odd and even list sizes. mergeRuns merges the sorted halves into a scratch buffer that mergeSort allocates once at the start, which avoids repeated memory allocation.
3. To stop binary search from using outdated data, I added a revision counter to ClinicManager, which increases whenever patient or appointment records change. PatientSorter stores the current revision with each sorted result. Before running binary search, SearchEngine checks that this revision still matches the live one. If it does not, the search falls back to linear search, and the status bar shows the technique used.
4. To stop the unwanted GUI events, I added an updating_ flag to PatientPanel. The flag is set while the panel updates the table or form itself, and the selection handler exits immediately while it is set.
5. I also made sure that undoing a deletion restores the patient's full state. DeletePatientAction saves the patient's details, all their appointments and their queue arrival time before the deletion takes place.

Part 3: Screenshots

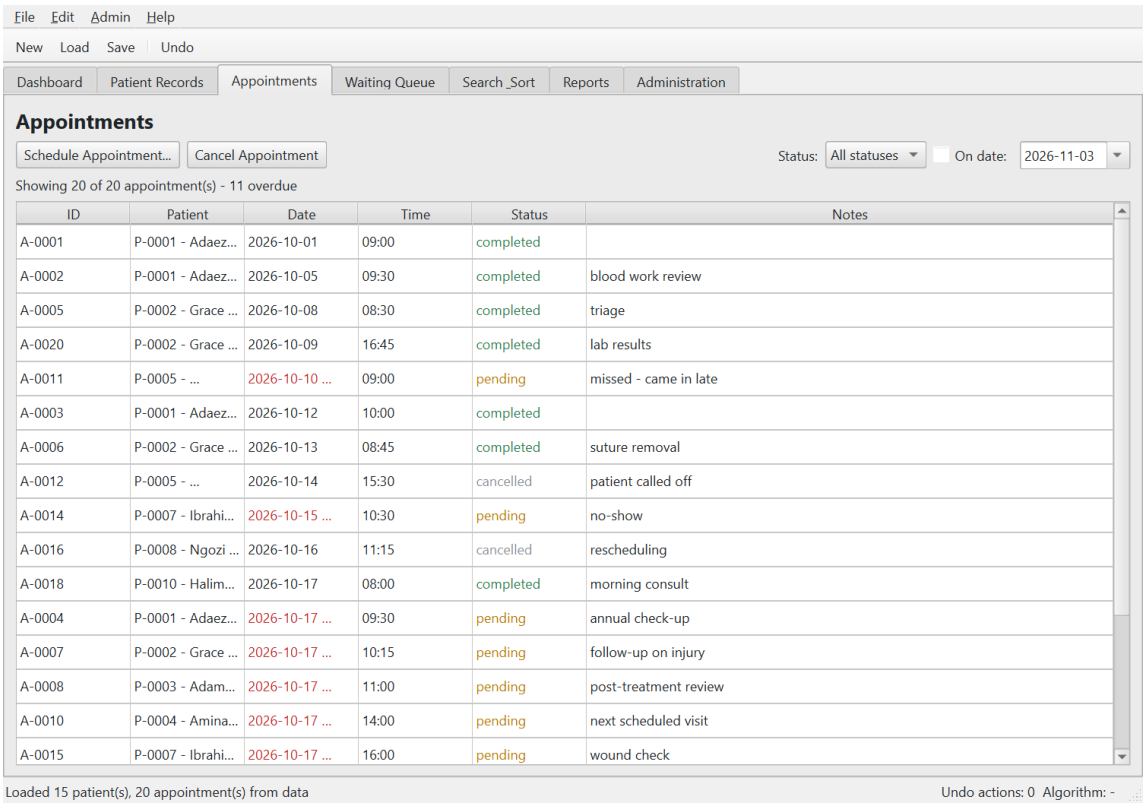
i. Main dashboard



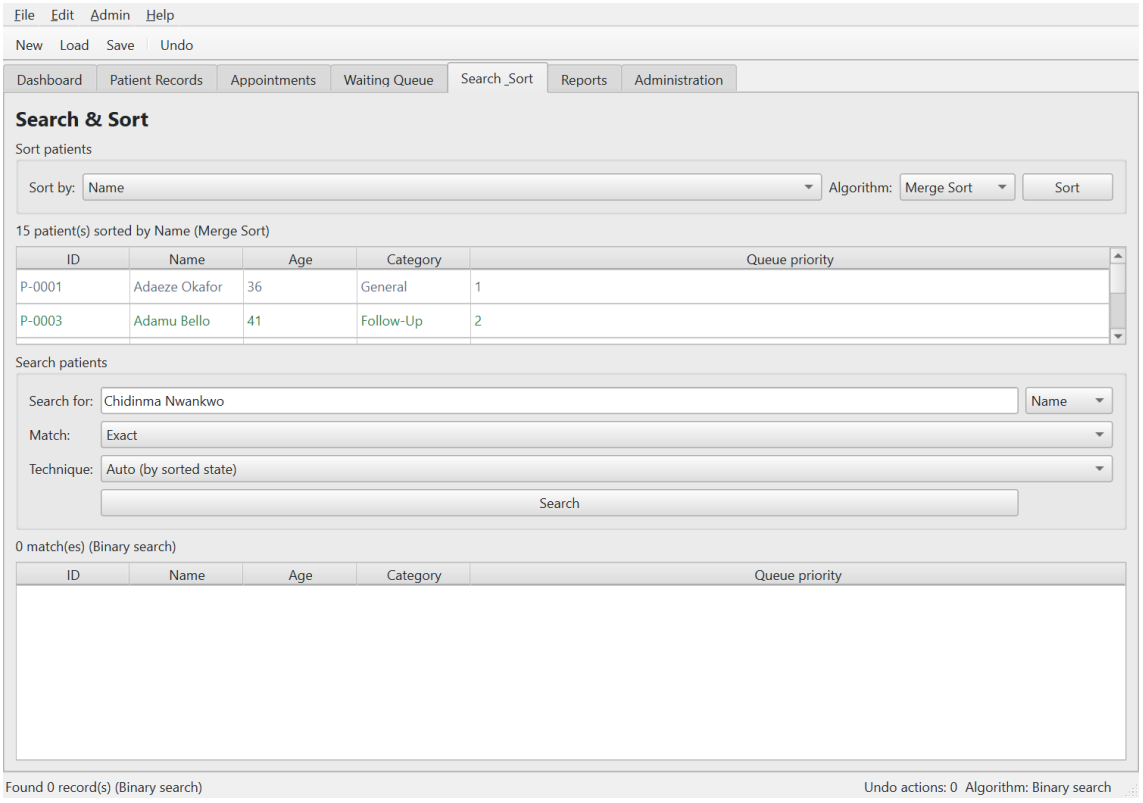
ii. Patient registration



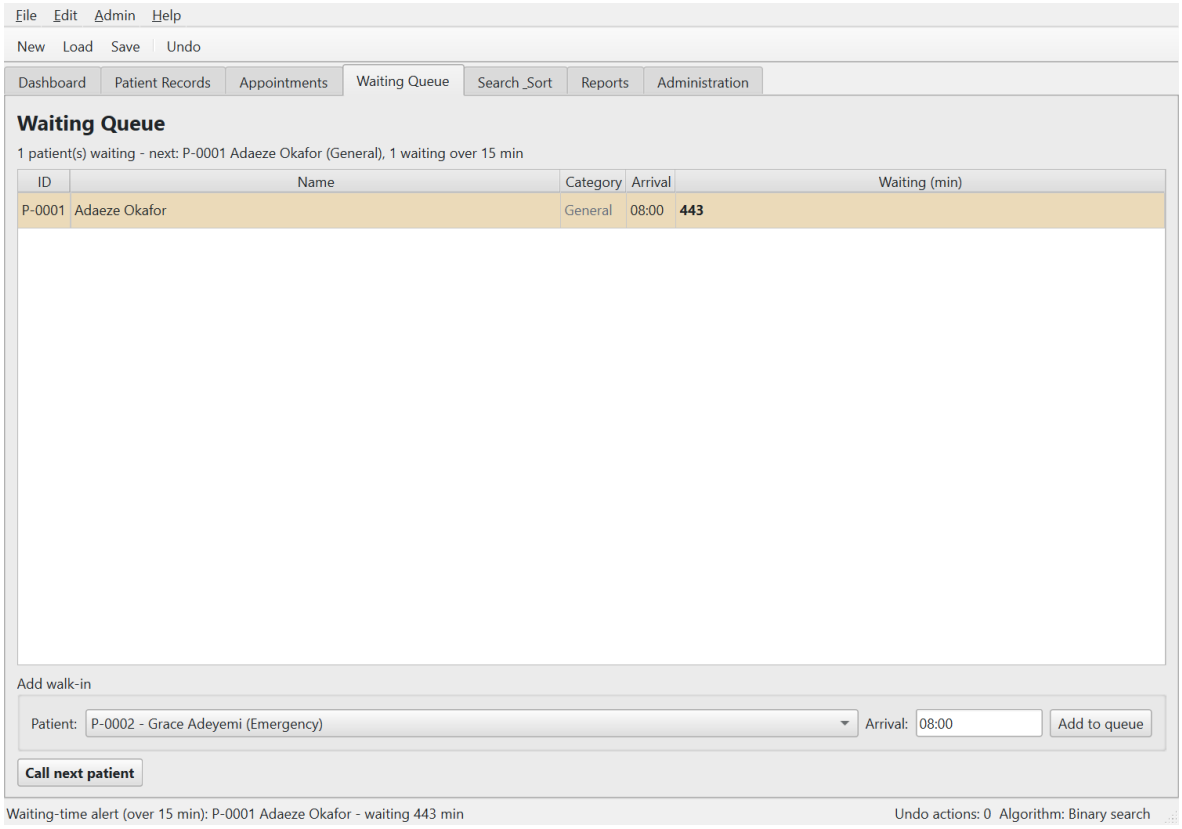
iii. Appointment management



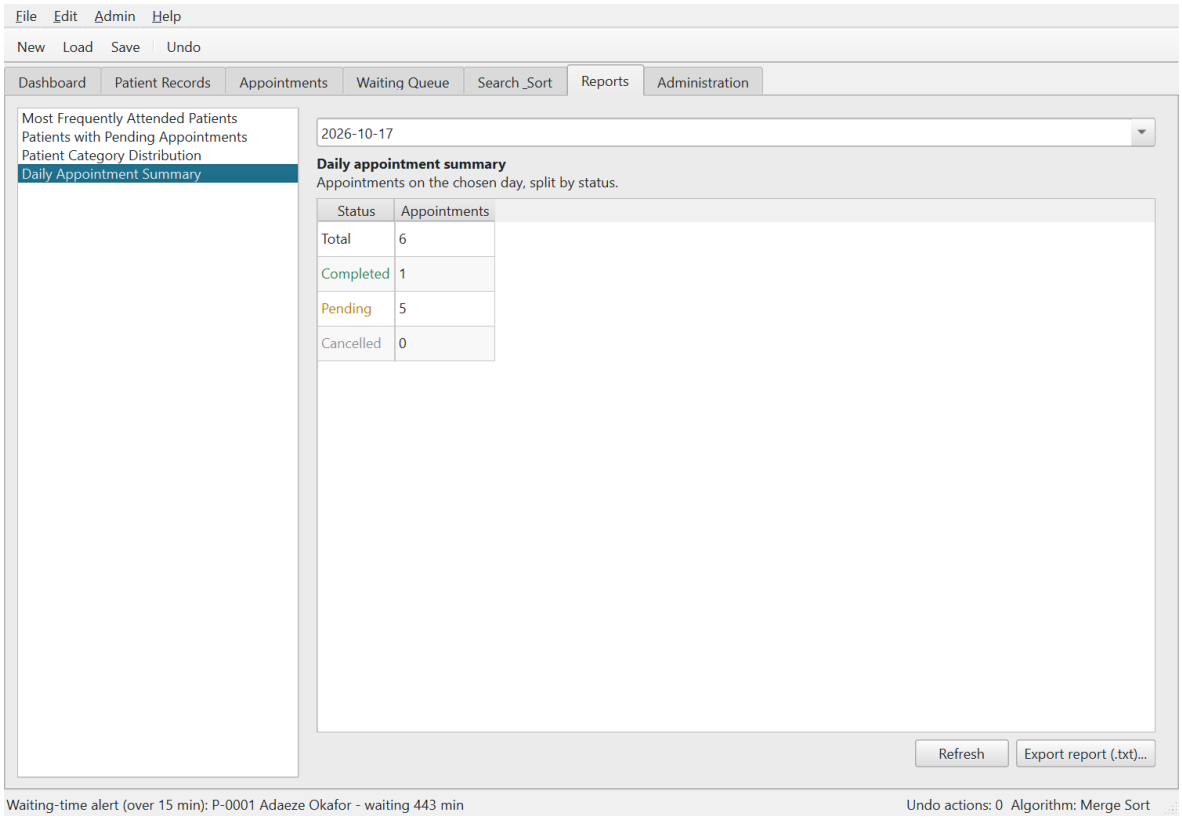
iv. Search/sort interface



v. Waiting queue



vi. Reports



Part 4: UML/Class Diagram – Drawn in draw.io

docs\submission\uml\scapms_class_diagram.drawio

